BOT Chain · chainId 677 · live on mainnet

AgentID — the agent registry that never sleeps.

An on-chain identity layer for AI agents: any wallet registers an agent with a name, category and description — permanent, owner-editable, and queryable by every app on BOT Chain.

agentid.druxamb.dev | github.com/DruxAMB/AgentID | scan.botchain.ai

The problem

AI agents have no portable identity. An agent's name, specialty and owner exist only inside whichever app happens to run it — there is no shared, credibly-neutral place to ask *who runs this agent and what does it do* that outlives any single frontend or company.

What AgentID is

On-chain registry. Agents are stored in contract storage on BOT Chain mainnet — name, category (writing, design, research, support, coding, data, other), description, owner, timestamps. No server, no database, no company to go offline.

Public directory. The dApp lists every registered agent with category filters and text search — read directly from chain, no wallet required to browse.

Owner-governed. The registering wallet can update its agent's description via `updateDescription`; nobody else can touch the row. Identity is self-sovereign by construction.

Permanently addressed. UUPS + ERC1967 proxy: the contract can be upgraded (features, fixes) while the address — the thing integrators reference — never changes.

Architecture

User wallet → AgentID dApp (Next.js + ethers) → ERC1967 Proxy `0xB9826C48e938a78B5E927bb98e827A7d5e2767C6` → Implementation `0x903eE4034f5a0c142f7A9468897fbc5464cd983c`

Registration and edits are signed transactions (`register`, `updateDescription`); directory reads are free calls (`agentsPage`, `agentCount`). Upgrade authority sits with the project owner wallet only.

Status & traction

Item	State
Contract	Deployed on BOT Chain mainnet — proxy `0xB982…76C6`, verified on scan.botchain.ai
Frontend	Live — agentid.druxamb.dev (register, filter, search, owner edit)
Code	Open source — github.com/DruxAMB/AgentID (MIT)
Stage	Mainnet live; registry open for first registrants

Why it fits BOT Chain

- **Native utility, not a wrapper** — BOT Chain's AI/DePIN positioning needs agent infrastructure; identity is the registry everything else (marketplaces, oracle consumers, reputation) builds on.
- **Real transaction surface** — every registration and edit is a user-paid mainnet transaction with a durable on-chain artifact.
- **Composable** — any contract can read the directory on-chain; other projects in the same ecosystem (proofs, tipping, badges) can resolve agent identity without permission.

Roadmap

- Rich agent metadata (avatar, endpoints, model/provider fields) via a linked metadata standard.
- Paged directory UI beyond the current 200-entry view, plus per-agent detail routes.
- Registry ↔ oracle integration: on-chain inference attribution for registered agents.
- Reputation hooks — tipping and badge issuance keyed to agent IDs.